

Politecnico di Torino

Department of Electronics and Telecommunications

Master's degree in Mechatronic Engineering



2025/2026

Sensors, Embedded Systems and Algorithms for Service Robotics

Report 2

Date, 22/12/2025

Group n° 212

Francesca Fornasier – S338688

Salvatore Loizzo – S344605

Daniele Torrero – S348210

Maria Teresa Tucci – S349416

OBJECTIVE

The aim of this laboratory activity is to use the information from both sensors, landmarks and odometry, to localize accurately the robot, and to develop a ROS 2 package that can permit to run the EKF algorithm on the robot itself.

DESIGN

For this lab a package named lab04_pkg has been created, in which there are:

- task0.py and task1.py, the Python files with the code of the two different exercises of the lab.
- utils.py, a Python file containing all the possible useful functions we need to run task1.py:
 - eval_jacobiane_Gt_e_Vt, which calculates
 - $Gt = gux.jacobian(Matrix([x, y, theta]))$
 - $Vt = gux.jacobian(Matrix([v, w]))$and then it converts them into high-performance numerical functions with lambdify, **eval_Gt** and **eval_Vt**.
 - sample_velocity_motion_model, which simulates the real movement of the robot, adds noise to the commands and implements the **probabilistic motion model**.
 - eval_jacobian_hux_Ht, which defines the **distance from the landmark** and the relative **bearing** with respect to the estimated state (it is used in the update phase of EKF) and calculates the Jacobian with respect to the state.
- metrics_from_rosbag_data.py, a file containing the functions to read the recorded data and compute the required metrics (Root Mean Square Error and Mean Absolute Error).
- ekf.py, the file in which there's the implementation of an Extended Kalman Filter encapsulated in the class RobotEKF. The class can manage: the robot's **estimated state**, the **error covariance matrix**, the **prediction** and **update phases** of the filter.

1.1 TASK 0

The code of this Task is structured to implement two fundamental probabilistic models in mobile robotics: the **velocity-based motion model** and the **landmark-based measurement model**. Both models are realized through dedicated functions that enable both stochastic sampling and analytical evaluation of the system's behavior.

- **Probabilistic motion model:** sample_velocity_motion_model

This function implements the **probabilistic velocity-based motion model**, representing the robot's kinematic evolution over time in response to specific velocity commands.

It receives as input parameters: the current **state** of the robot $[x, y, \theta]$, the input $\mathbf{u} = [v, \omega]$, **noise parameters** a , the time step dt .

Within the function, the primary step involves computing the noisy velocity components, v_hat and w_hat . The noise is modeled as a Gaussian distribution whose variance is proportional to the magnitude of the commanded velocities. Once the perturbed velocities are obtained, the function integrates the robot's trajectory using the kinematic model. Moreover if the angular velocity is negligible, a linear motion model is applied. Otherwise, a circular motion model (constant curvature) will be utilized.

The function returns a single sampled pose x_prime , representing one possible evolution of the robot's state. By invoking this function iteratively, as in the main script, a predictive distribution of future poses can be generated.

- **Motion Model Jacobians:** `compute_jacobians_vel_model`

This function calculates the **Jacobian matrices** of the motion model with respect to the state and the control input. Since the motion model is inherently non-linear, it must be linearized to be integrated into an Extended Kalman Filter (**EKF**). The function defines the motion model in symbolic form and subsequently computes:

- the Jacobian matrix with respect to the robot's state G_t
- the Jacobian matrix with respect to the control command V_t

These calculations have been done using SymPy to reduce the risk of mathematical errors.

- **Measurement Model:** `landmark_range_bearing_sensor`

This function simulates a **range-bearing sensor**, which measures the distance and the relative angle of a landmark with respect to the robot's pose. Given the robot's state and the landmark's coordinates, the function:

- Computes the ideal (deterministic) measurement
- Injects additive Gaussian noise into both the range and the bearing components
- Validates landmark visibility based on the field of view and the maximum sensing range.

- **Measurement Likelihood:** `landmark_model_prob`

Once a valid measurement is obtained, the code utilizes the `landmark_model_prob` function to compute the **measurement likelihood**. This is achieved by comparing the observed measurement with the expected measurement, derived from the robot's current state and the landmark's position.

The output is a probability value that quantifies the consistency of the observation with respect to the hypothesized state.

- **Pose Sampling from the Measurement Model:** `landmark_model_sample_pose`

This function implements **inverse sampling** from the measurement model: rather than computing a measurement from a known pose, it samples a pose based on a given measurement. Provided a landmark's coordinates and a range-bearing observation $[r, \phi]$, the function generates a robot pose that is consistent with that specific measurement.

- **Measurement Model Jacobian:** `compute_jacobian_sensor_model`

Finally, the code computes the Jacobian of the measurement model, yielding the matrix H_t . This matrix is essential for the measurement update step within the Extended Kalman Filter (EKF) framework.

1.2 TASK 1

This file implements a ROS 2 node (**EKF_ESTIMATE**) that performs robot state prediction at 20 Hz using a velocity motion model. It updates the state based on landmark measurements (range and bearing) and publishes the EKF estimate to the `/ekf` topic as a `nav_msgs/msg/Odometry` message. The EKF is instantiated using the following parameters:

```
self.robot=RobotEKF(dim_x=3, dim_u=2, eval_gux=sample_velocity_motion_model,eval_gt=self.eval_gt, eval_vt=self.eval_vt)
```

- **ROS 2 Topics:**

The node subscribes to the `/odom` topic to retrieve the robot's real-time velocity data, with

```
self.odom_sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)
```

The EKF prediction is done, instead, using a ROS timer. This approach ensures that the prediction step remains independent of incoming messages, executing at a constant rate:

```
self.timer = self.create_timer(0.05, self.prediction).
```

For the update step, the node subscribes to the `/landmarks` topic, which provides range and bearing measurements of the visible landmarks. Finally, the EKF estimate is published to the `/ekf` topic as an Odometry message.

- **Fetching robot velocities:** `odom_callback`

This function updates the control command used by EKF. It extracts the linear and angular velocities from the `/odom` message, representing the best available estimate of the robot's actual speed. It also includes numerical safeguards to prevent instabilities in the motion model when the angular velocity is near zero.

```
if abs(self.w) < 1e-6:
    self.w = 1e-6
```

- **Incorporating landmark measurements for state update:** `landmark_callback`

This function manages the state correction phase using landmark measurements. For each landmark, a measurement vector $z = np.array([[element.range], [element.bearing]])$ is constructed, representing the distance and bearing of the landmark relative to the robot and performing a data association check against the dictionary of known landmarks. Once all updates are completed, an Odometry message is generated, containing the EKF state estimate.

1.3 TASK 2

In Task 2, the state vector was expanded to include the robot's linear and angular velocities. This required modifying the motion model functions defined in Task 1. Furthermore, in addition to the landmark detection model, wheel encoder and IMU data were integrated to update and correct the state estimate. New observation functions and Jacobians were added to `utils2.py` to handle these sensors.

1.4 TASK 3

The Task 3 code does not introduce a new EKF; instead, it must: adapt the existing simulation EKF to **real-world conditions**, increase the **measurement noise parameters** and make the filter more **robust** to missing or noisy data.

EXPERIMENTS

Within the main script of **Task0**, the `sample_velocity_motion_model` function is invoked 500 times for each set of noise parameters. The aim is to observe how the **motion uncertainty** evolves in response to varying noise levels. The use of two distinct sets of alpha parameters is useful to highlight different uncertainty characteristics:

- In the first case, with $\alpha = [0.1, 0.1, 0.001, 0.02, 0.05, 0.05]$, linear motion uncertainty is predominant.
- In the second case, with $\alpha = [0.001, 0.001, 0.1, 0.2, 0.05, 0.05]$, there is a greater emphasis on angular motion uncertainty.

The resulting samples are then visualized, clearly demonstrating how the pose cloud disperses in different directions depending on the specific noise profile applied.

Repeating, instead, the operations of the function `landmark_model_sample_pose` 1000 times, we obtain a distribution of poses that represents the uncertainty on the position of the robot, taking into account what the sensor has observed.

The `plot_sampled_poses` function is subsequently employed to visualize this distribution, clearly illustrating the geometric and probabilistic implications of the model.

Regarding the **Task1** code, the function `groundTruth_callback` serves to print the robot's actual pose, enabling a qualitative comparison between the EKF estimate and the actual trajectory.

For both **simulation** and **real robot**, it is important to normalize the angular error and guarantee the bearing to remain between $[-\pi, \pi]$.

1.5 SIMULATION

In the simulation environment, three distinct trajectories are available:

1. **Ground Truth:** the actual trajectory of the simulated robot, serving as the ideal reference, smooth and consistent, drift-free, with no accumulation of numerical or sensor errors.
2. **Odometry:** the result of pure velocity integration (Dead Reckoning) shows a slight deviation from the Ground Truth. Although it exhibits some drift over time, the divergence remains low, and the estimated trajectory follows the real path.
3. **EKF:** a fused estimate combining the motion model and landmark observations, characterized by small local oscillations (due to measurement noise).

TASK1

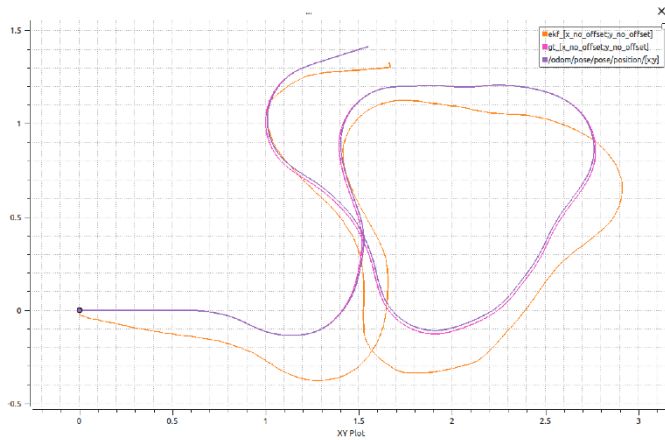


Figure 1 - EKF, ODOMETRY, GT trajectories in space

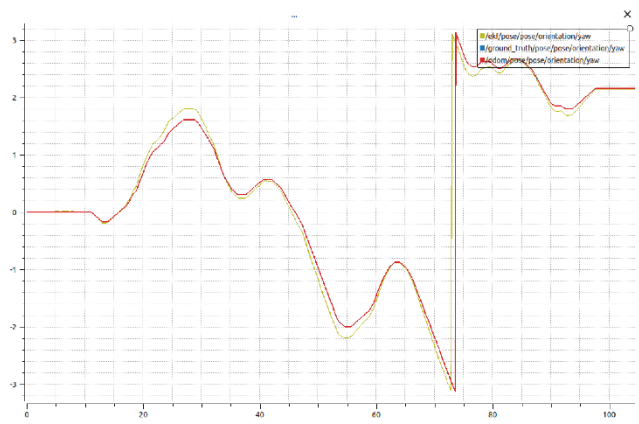


Figure 2 – EKF, ODOM -- Yaw (rad) w

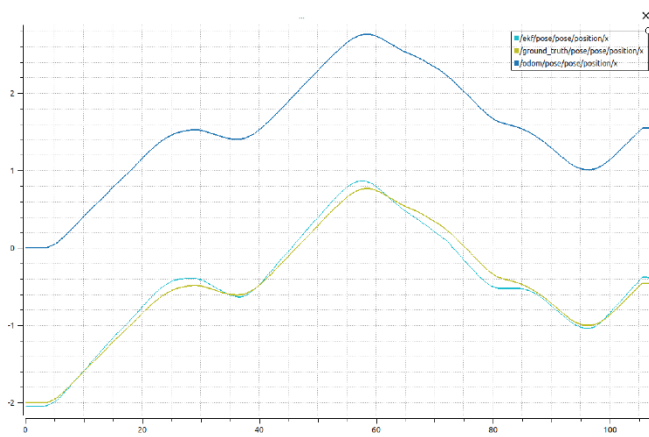


Figure 3 – EKF, ODOM, GT -- X wrt time

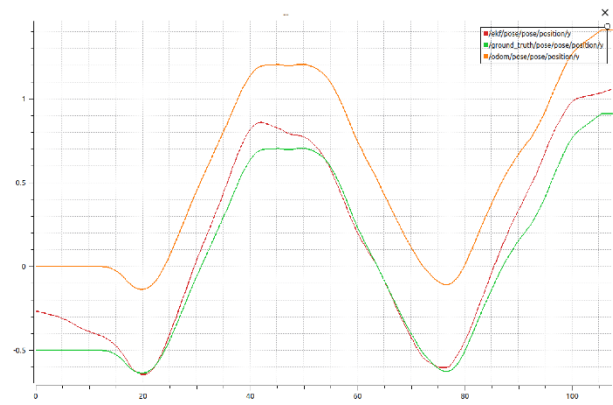


Figure 4 - EKF, ODOM, GT -- Y wrt time

TASK 2

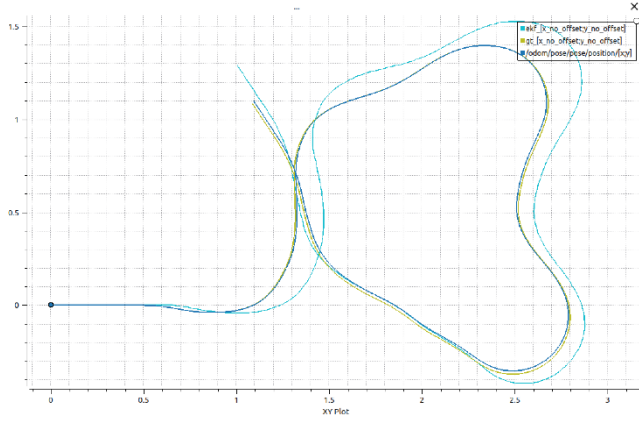


Figure 5 - EKF, ODOMETRY, GT trajectories in space – TASK1

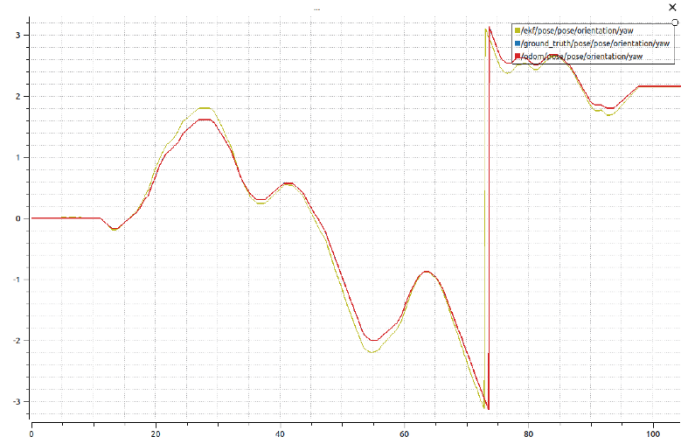


Figure 6 - EKF, ODOMETRY, GT trajectories in space – TASK1

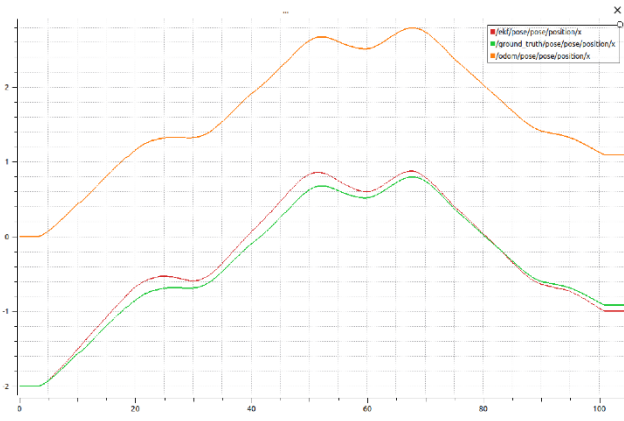


Figure 7 - EKF, ODOMETRY, GT trajectories in space – TASK1

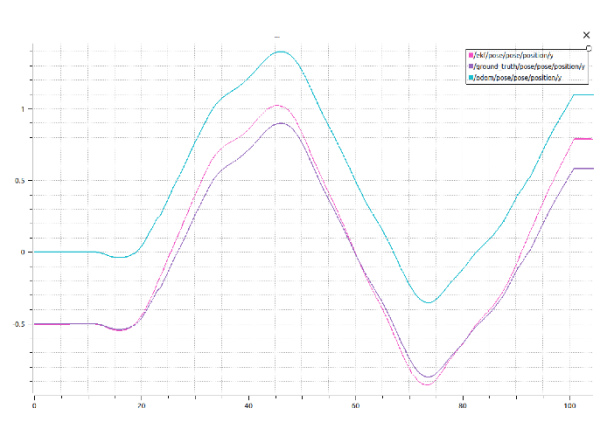


Figure 8 - EKF, ODOMETRY, GT trajectories in space – TASK1

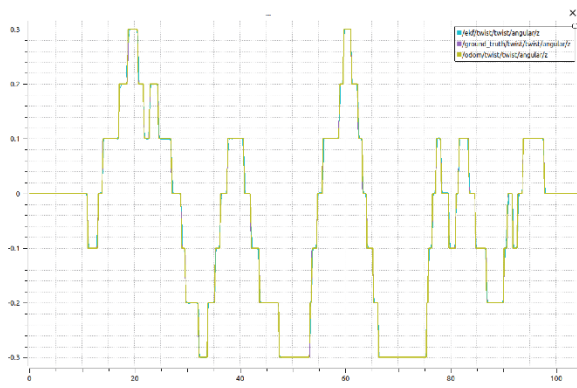


Figure 9 - EKF, ODOMETRY, GT trajectories in space – TASK1



Figure 10 - EKF, ODOMETRY, GT trajectories in space – TASK1

Metrics computations:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N |p_i - \hat{p}_i|^2}$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |p_i - \hat{p}_i|$$

	/odom → /ground_truth		/ekf → /ground_truth	
	TASK1	TASK2	TASK1	TASK2
RMSE_x [m]	2.0040	2.0038	0.6891	1.0626
RMSE_y [m]	0.5067	0.5076	0.9500	0.8392
RMSE_{theta} [rad]	0.0019	0.0018	1.5321	1.7353
MAE_x [m]	2.0040	2.0038	0.5163	0.8845
MAE_y [m]	0.5067	0.5076	0.7847	0.7066
MAE_{theta} [rad]	0.0013	0.0012	1.2483	1.4467

Commenting the results of the simulation of task 1 and 2 we could notice, as expected, through the graphical comparison and the coefficient of performances RMSE and MAE, that the trajectory of EKF is kind of improved. This is due to the fact that in task 2 more sensors were used to update the state of the robot in the Kalman Filter.

1.6 REAL ROBOT

On the physical robot, landmarks are not "perfect" as they are in simulation; instead, they are derived from the camera's visual perception. This implies higher noise on range and bearing measurements, potential missing landmarks (occlusions) and potential misidentification (incorrect IDs) or landmarks falling outside the field of view.

Moreover the topic to subscribe to now is **'/camera/landmarks'** (Real robot - Task 3), not anymore **'/landmarks'** (Simulation - Task 1).

A critical difference lies in the choice of measurement noise parameters, which determine – through Q_t - the trust to the sensor measurements. This choice is due to the fact that in the real world, sensors introduce both systematic and stochastic noise that is either absent or significantly reduced in simulation. So on the real robot the choice was to increase uncertainty of measurements: distance estimation is less accurate (0.8m vs. 0.3m) and bearing is noisier ($\pi/10$ vs. $\pi/24$).

	SIMULATION – TASK 1	REAL ROBOT (TASK 3)
<code>self.sigma_z</code>	<code>np.array([0.3, math.pi/24])</code>	<code>np.array([0.8, math.pi/10])</code>
<code>self.Qt</code>	<code>np.diag(self.sigma_z**2)</code>	<code>np.diag(self.sigma_z**2)</code>

From an EKF perspective, this results in:

- Reduced confidence in sensor measurements
- More conservative updates

- Greater weighting assigned to the prediction step.

While the motion model remains formally identical, the **noise parameters** must be retuned, because on the physical robot: velocities provided by `/odom` are derived from wheel encoders and these are generally more reliable than an ideal simulation relying on "perfect" control commands. The noise parameters are also used to compute the matrix M_t each iteration of prediction, since it's based on the noise parameters multiplied by the input velocities.

Finally, on the real robot, **ground truth** is often unavailable. The primary objective shifts to comparing `/odom` versus `/ekf` rather than evaluating against an absolute truth.

The trajectory of the real robot are:

1. **Odom:** pronounced drift visible after only a few meters, incorrect path curvature due to integration errors and cumulative orientation errors (heading drift).
2. **EKF:** higher stability compared to the raw odometry, reduced drift due to sensor fusion.

The filter let:

- Receive sensor data to correct the predictions
- Assigns confidence to the measurements by means of uncertainty matrixes.
- Performs more gradual corrections (smoother state updates).

By increasing the measurement noise covariance, the EKF becomes less sensitive to sensor noise, resulting in a smoother and more robust trajectory estimate.

While in figure 11 is depicted the plot of the trajectories in space x-y, in figure 12, 13 and 14 are depicted the component of trajectory with respect to time. It can be noticed, as before, that in figure 12 (yaw) there are some "jump" between $\pm\pi$. This is due to the fact that when the robot detects an angle of -181° , for instance, it automatically switches to 179°). In general the behavior of the trajectory is aligned to expectations.

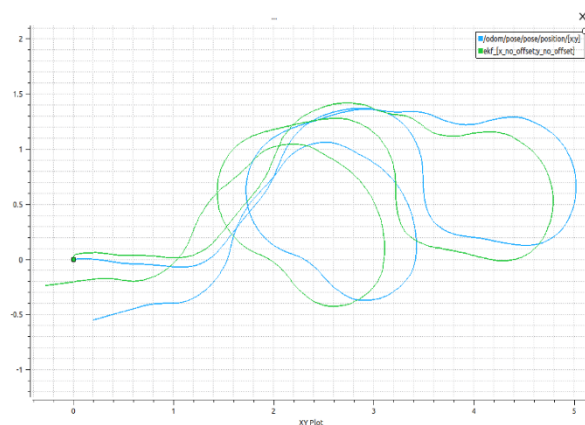


Figure 11 - EKF, ODOMETRY trajectories of real robot in

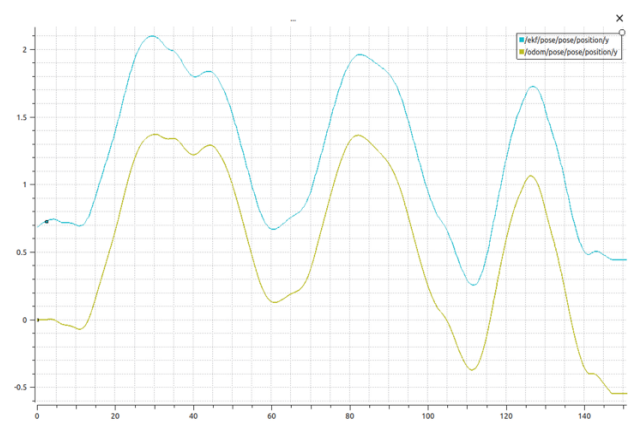


Figure 12- EKF, ODOM of the real robot -- Y wrt time

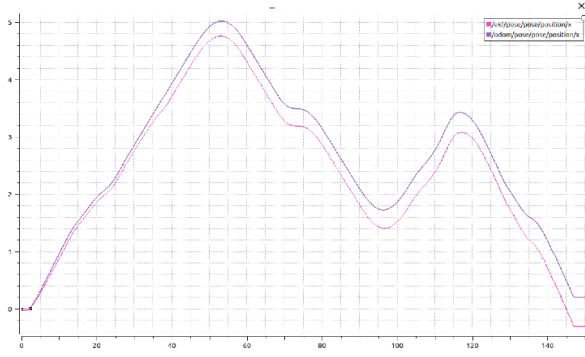


Figure 13 – EKF, ODOM of the real robot -- X wrt time

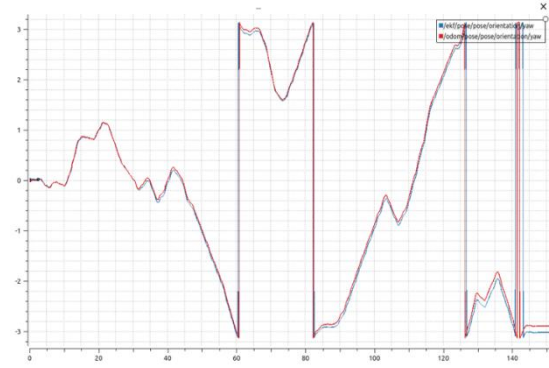
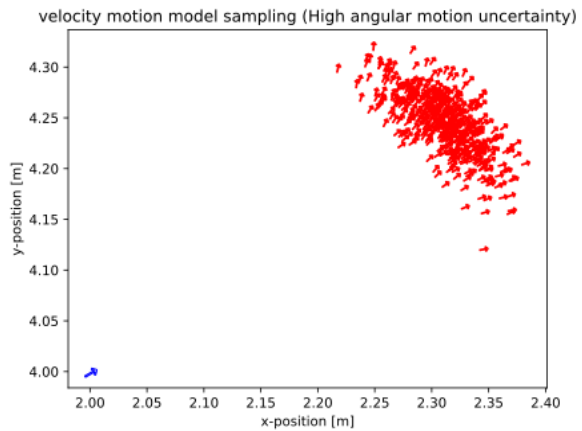
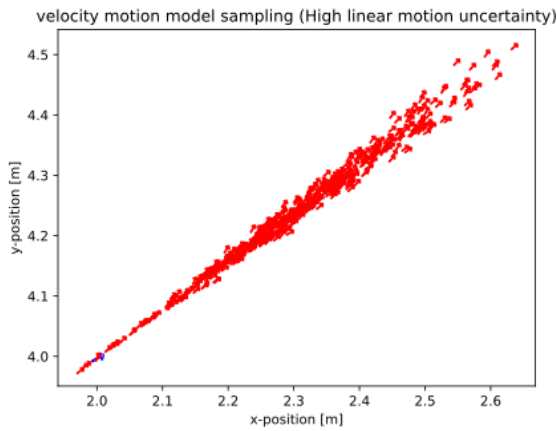


Figure 14– EKF, ODOM of the real robot -- Yaw (rad) wrt time

RESULTS AND DISCUSSION

The following plots show how uncertainty in the control inputs v and ω propagates to the robot's final pose. The plot on the right shows the initial pose (blue arrow) along with 500 samples of the predicted pose, obtained with values of α that highlight the uncertainty on the linear velocity. As expected, the resulting cloud of samples is significantly elongated along the direction of motion.

When the noise parameters are instead configured to model high variance in the angular velocity, the uncertainty propagates differently: the sample cloud spreads laterally and forms a characteristic arc-shaped pattern, reflecting the increased orientation uncertainty during motion.



Jacobian matrix obtained for the velocity motion model:

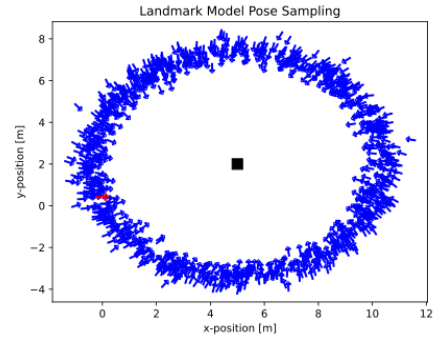
$$Gt = \begin{bmatrix} 1 & 0 & -\frac{v \cos(\theta)}{w} + \frac{v \cos(dtw + \theta)}{w} \\ 0 & 1 & -\frac{v \sin(\theta)}{w} + \frac{v \sin(dtw + \theta)}{w} \\ 0 & 0 & 1 \end{bmatrix}$$

$$Vt = \begin{bmatrix} -\frac{\sin(\theta)}{w} + \frac{\sin(dtw + \theta)}{w} & \frac{dtv \cos(dtw + \theta)}{w} + \frac{v \sin(\theta)}{w^2} - \frac{v \sin(dtw + \theta)}{w^2} \\ \frac{\cos(\theta)}{w} - \frac{\cos(dtw + \theta)}{w} & \frac{dtv \sin(dtw + \theta)}{w} - \frac{v \cos(\theta)}{w^2} + \frac{v \cos(dtw + \theta)}{w^2} \\ 0 & dt \end{bmatrix}$$

Linearized expression of the motion model:

$$g(u_t, x_t) \approx g(u_t, \mu_{t-1}) + G_t(x_{t-1} - \mu_{t-1})$$

This plot on the right shows the sampling of possible robot poses (blue arrows) generated from a single landmark detection. Since the robot measures only the distance to the landmark (represented as a black square) and the relative bearing with respect to its heading, its position is not uniquely determined. As a result, the set of feasible poses is distributed along a ring-shaped region centred on the landmark.



Jacobian matrix obtained for the measurement model:

$$Ht = \begin{bmatrix} \frac{-m_x + x}{\sqrt{(m_x - x)^2 + (m_y - y)^2}} & \frac{-m_y + y}{\sqrt{(m_x - x)^2 + (m_y - y)^2}} & 0 \\ -\frac{-m_y + y}{(m_x - x)^2 + (m_y - y)^2} & -\frac{m_x - x}{(m_x - x)^2 + (m_y - y)^2} & -1 \end{bmatrix}$$

Linearized expression of the measurement model:

$$h(x_t) \approx h(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t)$$